Шаблон отчёта по лабораторной работе

Простейший вариант

Дмитрий Сергеевич Кулябов

Содержание

1 Цель работы **6**

2 Задание **7**

3 Теоретическое введение **8**

4 Выполнение лабораторной работы **9**
4.1 Модель SIR 9

5 Модель Лотки–Вольтерры **40**
5.1 МОДЕЛЬ ЛОТКИ-ВОЛЬТЕРРЫ (ХИЩНИК-ЖЕРТВА) 54

6 Основной цикл вычислений (весь вывод закомментирован) **56**

7 Сравнительный анализ **58**

8 Графики сравнения **59**

9 Выводы **60**

Список литературы **61**

Список иллюстраций

4.1 setup_project 10
4.2 setup_project 10
4.3 выполнение 10
4.4 ошибка 18
4.5 установка 18
4.6 задача решается 19
4.7 Форматысоздаются 19
4.8 notebook работает 20

5.1 notebook работает 53

Список таблиц

```
# □□□□□□□□□□□□□ □□□□□□□□□□□□□ □□□ □□□□□□
include("../project/setup_report.jl")

# □□□□□□□□□□ □□□□□□ □□□□□□
using DifferentialEquations
using SimpleDiffEq
using Tables
using DataFrames
using StatsPlots
using LaTeXStrings
using Plots
using BenchmarkTools
using CSV

function sir_ode!(du, u, p, t)
    (S, I, R) = u
    (□, c, □) = p
    N = S + I + R
    @inbounds begin
        du[1] = -□ * c * I / N * S
        du[2] = □ * c * I / N * S - □ * I
        du[3] = □ * I
    end
```

```
    nothing
end

println("SIR □□□□□□□ □□□□□□□□□")
```

□□□□□□□□□□ □□□□□□: C:\Users\bogda\work\study\2026-1\2026-1--study--simulationmodeling\labs\lab02\project

□□□□□□ □□□□□□□□□□□: project

SIR □□□□□□□ □□□□□□□□□

1 Цель работы

Исследовать модели основных методов эпидемиологии на примере SIR и LV. Также научиться переводить код в литературный вид и получить опыт пользования языком julia

2 Задание

Создать рабочий каталог для кода. Установить необходимые пакеты. Выполнить предложенный код. Преобразовать код в литературный стиль. Сгенерировать из литературного кода: чистый код; jupyter notebook; документацию в формате Quarto. Выполнить код из jupyter notebook. Интегрировать документацию в формате Quarto в отчёт. Добавить в код в литературном стиле вычисление для набора параметров. Сгенерировать из литературного кода с параметрами: чистый код; jupyter notebook; документацию в формате Quarto. Выполнить код из jupyter notebook с параметрами. Интегрировать документацию с параметрами в формате Quarto в отчёт.

3 Теоретическое введение

Модель SIR делит всю популяцию на три взаимосвязанные группы (компартменты), что отражено в её названии:

S — Susceptible (Восприимчивые): люди, которые не болели, не имеют иммунитета и могут заразиться.

I — Infectious (Инфицированные/Заразные): люди, которые в данный момент больны и могут передавать инфекцию.

R — Recovered (Выздоровевшие/Удаленные): люди, которые переболели и приобрели иммунитет (или умерли). Они больше не участвуют в процессе передачи.

Основная цель модели: не предсказать судьбу конкретного человека, а понять общую динамику эпидемии — будет ли она разрастаться, как быстро, сколько людей в итоге переболеет, как влияют карантинные меры.

Модель Лотки-Вольтерры — это фундаментальная математическая модель в экологии, описывающая динамику взаимодействия двух видов: хищников и жертв. Она была независимо предложена в 1920-х годах:

Альфредом Лоткой (1925) для химических реакций [3,4].

Витторио Вольтеррой (1926) для объяснения колебаний улова рыбы в Адриатическом море [5,6].

Модель демонстрирует, как даже простая система взаимодействий может порождать сложные колебательные режимы, объясняя циклические изменения численности в природных экосистемах.

4 Выполнение лабораторной работы

4.1 Модель SIR

Создаю пространство для выполнения лабораторной. Для этого создаю файл setup_report(рис. 4.1). :

```
#!/usr/bin/env julia
# setup_project.jl

using Pkg
Pkg.add("DrWatson")
using DrWatson

project_name = "project"
initialize_project(project_name; authors="Bogdan", git=false)

println("□ □□□□□□ □□□□□□: ", project_name)
println("□ □□□□□□□□□ □ □□□□□□□□□□: cd ", project_name)
```

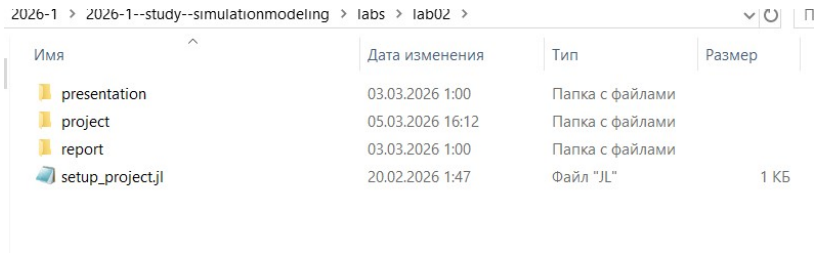


Рисунок 4.1: setup_project

Выполняю setup_project.jl(рис. 4.1).

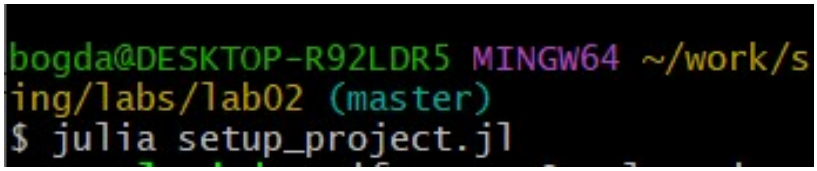


Рисунок 4.2: setup_project

Перехожу в создавшуюся папку скриптс, куда создаю ноовый скрипт и вы полняю его(рис. 4.3):

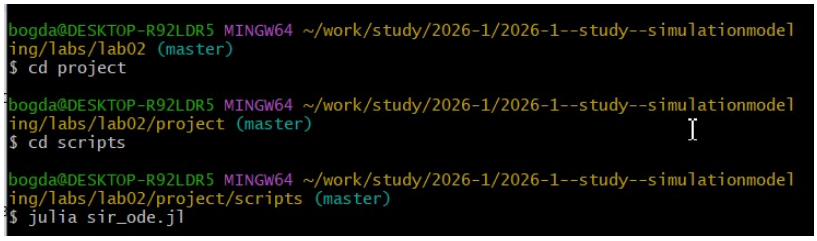


Рисунок 4.3: выполнение

сам код:

```
using DrWatson
@quickactivate "project"




using DifferentialEquations
```

```julia
using SimpleDiffEq
using Tables
using DataFrames
using StatsPlots
using LaTeXStrings  # □□□ □□□□□□□□□ □□□□□□□□□□□ □□□□□□ □□ □□□□□□□□□
using Plots
using BenchmarkTools

script_name = splitext(basename(PROGRAM_FILE))[1]
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))

# ## □□□□□□ □□□□□-□□□□□□□□□□

function sir_ode!(du, u, p, t)
    (S, I, R) = u
    (□, c, □) = p
    N = S + I + R
    @inbounds begin
        du[1] = -□ * c * I / N * S
        du[2] = □ * c * I / N * S - □ * I
        du[3] = □ * I
    end
    nothing
end

# □□□□□□□□□ □□□□□□
□t = 0.1
```

```julia
tmax = 40.0
tspan = (0.0, tmax)
u0 = [990.0, 10.0, 0.0]  # S, I, R
p = [0.05, 10.0, 0.25]   # β, c, γ

# 基本参数 基本再生数 基本传染病参数 基本参数
R0 = (p[2] * p[1]) / p[3]  # R₀ = (c * β) / γ

# 求解常微分 方 求解常微分 求解常微分
prob_ode = ODEProblem(sir_ode!, u0, tspan, p)
sol_ode = solve(prob_ode, dt = 0.1)

# 转换为数据 数据帧 为 DataFrame
df_ode = DataFrame(Tables.table(sol_ode'))
rename!(df_ode, ["S", "I", "R"])
df_ode[!, :t] = sol_ode.t
df_ode[!, :N] = df_ode.S + df_ode.I + df_ode.R  # 计算 总人口数量 总人口

# 基本 基本传染病 基本参数
println("基本传染病 基本参数 SIR:")
println("β (传染率 感染率) = ", p[1])
println("c (接触率 平均 感染率) = ", p[2])
println("γ (恢复率 康复率) = ", p[3])
println("R₀ = c * β / γ = ", round(R0, digits=3))
println("平均感染 传染期间接触人数 平均感染 = ", round(1/p[3], digits=2), "
println("初始条件 初始人数: S₀ = ", u0[1], ", I₀ = ", u0[2], ", R₀ = ",

# 1. 时间序列 图表: 时间序列 图表 图表 图表
```

```
plt1 = @df df_ode plot(:t,
    [:S :I :R],
    label=[L"S(t)" L"I(t)" L"R(t)"],
    xlabel="□□□□□, □□□",
    ylabel="□□□□□□□□□□ □□□□□",
    title="□□□□□□ SIR: □□□□□□□□ □□□□□□□□",
    linewidth=2,
    legend=:right,
    grid=true,
    size=(800, 500))

# □□□□□□□□□□ □□□□□□□□□ □ □□□□□□□□□□□
annotate!(plt1, maximum(df_ode.t) * 0.7, maximum(df_ode.N) * 0.8,
    text("□□□□□□□□□:\n□ = $(p[1])\nc = $(p[2])\n□ = $(p[3])\nR₀ = $(rc
    8, :left))

# □□□□□□ □□□□□□ □□□□□□□□□□□□□□ (I)
plt2 = @df df_ode plot(:t, :I,
    label=L"I(t)",
    xlabel="□□□□□, □□□",
    ylabel="□□□□□□□□□□ □□□□□□□□□□□□□□",
    title="□□□□□□□□ □□□□□ □□□□□□□□□□",
    color=:red,
    linewidth=2,
    fill=(0, 0.3, :red),
    grid=true,
    size=(800, 400))
```

```
# □□□□□□□ □□□□ □□□□□□□□
peak_idx = argmax(df_ode.I)
peak_time = df_ode.t[peak_idx]
peak_value = df_ode.I[peak_idx]
vline!(plt2, [peak_time], color=:black, linestyle=:dash, label=false
annotate!(plt2, peak_time, peak_value * 1.05,
    text("□□□: $(round(peak_value, digits=1)) □□ $(round(peak_time, d
    8, :top))

# □□□□□□ □ □□□□□□□□□□□□□□□ □□□□□□□□ (□□□ □□□□□□□ □□□□□□□□□□□□□□□□□ □□□□□)
plt3 = @df df_ode plot(:t, :I,
    label=L"I(t)",
    xlabel="□□□□□, □□□",
    ylabel="□□□□□□□□□□ □□□□□□□□□□□□□□ (□□□. □□□□□□□)",
    title="□□□□□□□□□□□□□□□□ □□□□ (□□□. □□□□□)",
    yscale=:log10,
    color=:red,
    linewidth=2,
    grid=true,
    size=(800, 400))

# □□□□□□ □□□□□ □□□□□□□□□ (□ □□□□□□□□□)
plt4 = @df df_ode plot(:t,
    [:S :I :R] ./ df_ode.N .* 100,
    label=[L"S(t)/N" L"I(t)/N" L"R(t)/N"],
    xlabel="□□□□□, □□□",
    ylabel="□□□□ □□□□□□□□□, %",
    title="□□□□□□□□ □□□□□□□□ (□ □□□□□□□□□)",
```

```
    linewidth=2,
    legend=:right,
    grid=true,
    size=(800, 500))

# □□□□□□□□□□□□□□ □□□□□ □□□ □□□□□□ □□□□□□□□□□□□□ □□□□□□□□□□
if R0 > 1
    herd_immunity_threshold = (1 - 1/R0) * 100
    hline!(plt4, [herd_immunity_threshold], color=:purple, linestyl
        label="□□□□□ □□□□□□□□□□□□□ □□□□□□□□□□ ($(round(herd_immunity_thres
        linewidth=1.5)
end

# □□□□□□□ □□□□□□□ (I vs S)
plt5 = plot(df_ode.S, df_ode.I,
    label="□□□□□□□ □□□□□□□□□□",
    xlabel=L"S(t)",
    ylabel=L"I(t)",
    title="□□□□□□□ □□□□□□□ SIR □□□□□□",
    color=:blue,
    linewidth=2,
    grid=true,
    size=(800, 500),
    legend=:topright)

# □□□□□□□□□□ □□□□□□□ □□□□□□□□□□□
for i in 1:50:length(df_ode.S)-1
    plot!(plt5, [df_ode.S[i], df_ode.S[i+1]], [df_ode.I[i], df_ode.
```

```
        arrow=:closed, color=:blue, alpha=0.5, label=false)
end

# □□□□□□ Rₑ - □□□□□□□□□□□□ □□□□□□□□□□□□□□□ □□□□□
df_ode[!, :Re] = R0 .* df_ode.S ./ df_ode.N
plt6 = @df df_ode plot(:t, :Re,
    label=L"R_e(t)",
    xlabel="□□□□□, □□□",
    ylabel=L"R_e",
    title="□□□□□□□□ □□□□□□□□□□□□ □□□□□□□□□□□□□□□ □□□□□",
    color=:green,
    linewidth=2,
    grid=true,
    size=(800, 400))

# □□□□□□□□□□□□□□□ □□□□□ □□ □□□□□□ 1
hline!(plt6, [1.0], color=:red, linestyle=:dash, label="□□□□□ □□□□□□□□

# □□□□□□□ □□□□□□□, □□□□□ Rₑ □□□□□□□□□□ < 1
cross_idx = findfirst(x -> x < 1, df_ode.Re)
if !isnothing(cross_idx) && cross_idx > 1
    cross_time = df_ode.t[cross_idx]
    vline!(plt6, [cross_time], color=:black, linestyle=:dash, label
    annotate!(plt6, cross_time, 1.2,
        text("Rₑ<1 □ $(round(cross_time, digits=1)) □□□", 8, :left))
end

# □□□□□□□□□□ □□□□□□ □□□□ □□□□□□ □ □□□□□ □□□□□□
```

```
plt7 = plot(layout=(2, 3), size=(1200, 800))

# □□□□□□□ □□□
plot!(plt7[1], df_ode.t, df_ode.S, label=L"S(t)", color=1, linewidt
plot!(plt7[2], df_ode.t, df_ode.I, label=L"I(t)", color=2, linewidt
plot!(plt7[3], df_ode.t, df_ode.R, label=L"R(t)", color=3, linewidt

# □□□□□□ □□□
plot!(plt7[4], df_ode.t, df_ode.I, label=L"I(t)", color=2, linewidt
    yscale=:log10, title="□□□. □□□□□□□")
plot!(plt7[5], df_ode.S, df_ode.I, label=false, color=4, linewidth=
    title="□□□□□□ □□□□□□□", xlabel=L"S", ylabel=L"I")
plot!(plt7[6], df_ode.t, df_ode.Re, label=L"R_e", color=:green, lin
    title=L"R_e(t)", hline=[1.0], linestyle=:dash, linecolor=:red)

# □□□□□□□□□□ □□□□□□□□

savefig(plt1, plotsdir(script_name, "sir_main.png"))
savefig(plt2, plotsdir(script_name, "sir_infected.png"))
savefig(plt3, plotsdir(script_name, "sir_log_scale.png"))
savefig(plt4, plotsdir(script_name, "sir_percentages.png"))
savefig(plt5, plotsdir(script_name, "sir_phase_portrait.png"))
savefig(plt6, plotsdir(script_name, "sir_effective_R.png"))
savefig(plt7, plotsdir(script_name, "sir_panel.png"))

# □□□□□□□□ □□□ □□□□□□ □□□□□□□□□□□□□□□□□□
println("\n□□□□□□□□ □□□□□□□:")
@benchmark solve(prob_ode, dt = □t)
```

```
# □□□□□□□□□□□□□□ □□□□□□
println("\n=== □□□□□□ □□□□□□□□□□□ ===")
println("□□□□□ □□□□□□□□□□□ □□□□□□□□□ (□□□□□□□□): N = ", round(df_ode.N[1],
println("□□□□□□□ □□□□□ □□□□□□□□□□: I_max = ", round(peak_value, digits=1
println("□□□□□ □□□□□□□□□□ □□□□: t_peak = ", round(peak_time, digits=1),
println("□□□□□□□□ □□□□□ □□□□□□□□□□□□: R(∞) = ", round(df_ode.R[end], digi
println("□□□□ □□□□□□□□□□□□: ", round(df_ode.R[end]/df_ode.N[1]*100, dig

if R0 > 1
    println("\n□□□□□□□□□□□□□ □□□□□□:")
    println("  - □□□□□ □□□□□□□□□□□□□ □□□□□□□□□□: ", round((1-1/R0)*100, di
    println("  - □□□□□□□□□□□□□ □□□ □□□ S/N = 1/R₀ = ", round(1/R0, digit
end
```

Не хватает библиотеки с дифференциальными уравнениями(рис. 4.4)

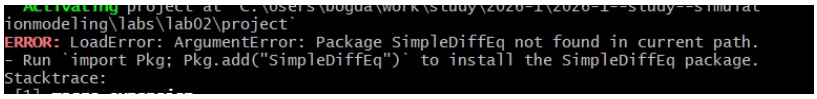


Рисунок 4.4: ошибка

Докачиваю необходимую библиотеку. (на самом деле ещё пару библиотек пришлось установить)(рис. 4.5)

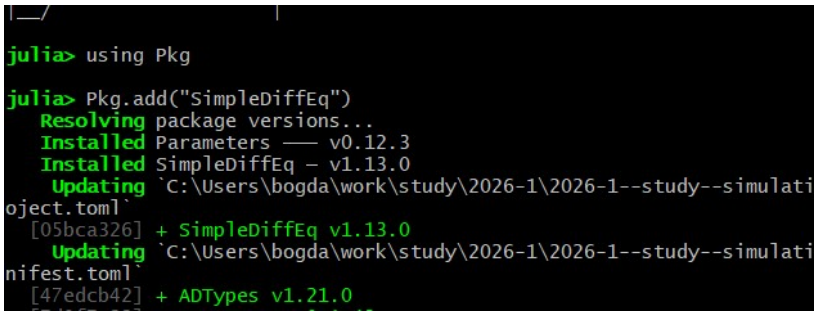


Рисунок 4.5: установка

Выполняю скрипт - всё рабоатет (рис. 4.6)

```
bogda@DESKTOP-R92LDR5 MINGW64 ~/work/study/2026-1/2026-1
 (master)
$ julia --project=. scripts/sir_ode.jl
Параметры модели SIR:
β (вероятность заражения) = 0.05
c (среднее число контактов) = 10.0
γ (скорость выздоровления) = 0.25
R₀ = c * β / γ = 2.0
Средняя продолжительность болезни = 4.0 дней
Начальные условия: S₀ = 990.0, I₀ = 10.0, R₀ = 0.0

Бенчмарк решения:

=== АНАЛИЗ РЕЗУЛЬТАТОВ ===
Общая численность популяции (контроль): N = 1000.0
Пиковое число зараженных: I_max = 154.8
Время достижения пика: t_peak = 19.1 дней
Итоговое число переболевших: R(∞) = 775.7
Доля переболевших: 77.6%

Теоретический анализ:
  - Порог коллективного иммунитета: 50.0%
  - Теоретический пик при S/N = 1/R₀ = 0.5
```

Рисунок 4.6: задача решается

Создаю остальные производные форматы (рис. 4.7)

```
bogda@DESKTOP-R92LDR5 MINGW64 ~/work/study/2026-1/2026-1--study--simulationmodeling/labs/lab02/project
 (master)
$ julia --project=. scripts/tangle.jl scripts/sir_ode.jl
Генерация из: scripts/sir_ode.jl
[ Info: generating plain script file from `C:\Users\bogda\work\study\2026-1\2026-1--study--simulationm
odeling\labs\lab02\project\scripts\sir_ode.jl`
[ Info: writing result to `C:\Users\bogda\work\study\2026-1\2026-1--study--simulationmodeling\labs\lab
02\project\scripts\sir_ode\sir_ode.jl`
  ✓ Чистый скрипт: C:\Users\bogda\work\study\2026-1\2026-1--study--simulationmodeling\labs\lab02\proje
ct\scripts\sir_ode/sir_ode.jl
[ Info: generating markdown page from `C:\Users\bogda\work\study\2026-1\2026-1--study--simulationmodel
ing\labs\lab02\project\scripts\sir_ode.jl`
[ Info: writing result to `C:\Users\bogda\work\study\2026-1\2026-1--study--simulationmodeling\labs\lab
02\project\markdown\sir_ode\sir_ode.qmd`
  ✓ Quarto: C:\Users\bogda\work\study\2026-1\2026-1--study--simulationmodeling\labs\lab02\project\mark
down\sir_ode/sir_ode.qmd
[ Info: generating notebook from `C:\Users\bogda\work\study\2026-1\2026-1--study--simulationmodeling\l
abs\lab02\project\scripts\sir_ode.jl`
[ Info: writing result to `C:\Users\bogda\work\study\2026-1\2026-1--study--simulationmodeling\labs\lab
02\project\notebooks\sir_ode\sir_ode.ipynb`
  ✓ Notebook: C:\Users\bogda\work\study\2026-1\2026-1--study--simulationmodeling\labs\lab02\project\no
tebooks\sir_ode/sir_ode.ipynb

Готово! Все файлы созданы.

bogda@DESKTOP-R92LDR5 MINGW64 ~/work/study/2026-1/2026-1--study--simulationmodeling/labs/lab02/project
```

Рисунок 4.7: Форматысоздаются

Запускаю notebook (рис. 4.8)

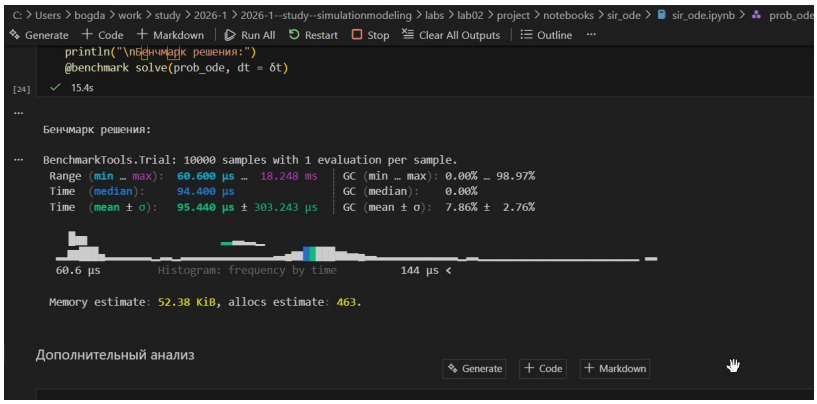


Рисунок 4.8: notebook работает

```
using DrWatson
@quickactivate "project"




using DifferentialEquations
using SimpleDiffEq
using Tables
using DataFrames
using StatsPlots
using LaTeXStrings  # □□□ □□□□□□□□□ □□□□□□□□□□□ □□□□□□ □□ □□□□□□□□□
using Plots
using BenchmarkTools

script_name = splitext(basename(PROGRAM_FILE))[1]
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))

function sir_ode!(du, u, p, t)
```

```
    (S, I, R) = u
    (□, c, □) = p
    N = S + I + R
    @inbounds begin
        du[1] = -□ * c * I / N * S
        du[2] = □ * c * I / N * S - □ * I
        du[3] = □ * I
    end
    nothing
end
```

```
┌ Warning: DrWatson could not find find a project file by recursively cl
□ (given dir: C:\Users\bogda)
└ @ DrWatson C:\Users\bogda\.julia\packages\DrWatson\2QF5p\src\proje
```

```
sir_ode! (generic function with 1 method)
```

Параметры модели

```
□t = 0.1
tmax = 40.0
tspan = (0.0, tmax)
u0 = [990.0, 10.0, 0.0]  # S, I, R
p = [0.05, 10.0, 0.25]   # □, c, □
```

```
3-element Vector{Float64}:
  0.05
 10.0
  0.25
```

Расчет базового репродуктивного числа

```
R0 = (p[2] * p[1]) / p[3]  # R₀ = (c * ⍺) / ⍵
```

```
2.0
```

Создание и решение задачи

```
prob_ode = ODEProblem(sir_ode!, u0, tspan, p)
sol_ode = solve(prob_ode, dt = ⍥t)
```

```
retcode: Success
Interpolation: 3rd order Hermite
t: 15-element Vector{Float64}:
  0.0
  0.1
  0.5331242537191838
  1.3919008491040001
  2.6102120755226386
  4.1811820758323055
  6.180123810566044
  8.664951930903612
 11.691425331813486
 15.279370667110317
 19.082963490744028
 23.423425193442526
 28.42949373098617
 33.298779173639595
 40.0
u: 15-element Vector{Vector{Float64}}:
 [990.0, 10.0, 0.0]
 [989.4990153200853, 10.247898072276143, 0.25308660763853114]
```

```
[987.1852964117763, 11.391111426852984, 1.4235921613706284]
[981.8312125227972, 14.026022042698543, 4.142765434504278]
[972.1392310589528, 18.757815561654184, 9.102953379393014]
[954.9905500491583, 27.007702518267596, 18.001747432574035]
[923.0706717929863, 41.9297677983272, 34.999560408686385]
[862.6705810342163, 68.49345419063425, 68.83596477514936]
[754.3022993851441, 109.7418618540929, 135.95583876076284]
[595.1014268847738, 150.4136326448068, 254.48494047041933]
[442.01411791013015, 154.80694988491862, 403.1789322049512]
[326.89511674939587, 119.06860394684871, 554.0362793037554]
[258.50852421889647, 70.1105219475306, 671.3809538335728]
[227.58338337085723, 37.32857063157995, 735.0880459975627]
[209.8412140201436, 14.473644966455813, 775.6851410134005]
```

Подготовка данных в DataFrame

```
df_ode = DataFrame(Tables.table(sol_ode'))
rename!(df_ode, ["S", "I", "R"])
df_ode[!, :t] = sol_ode.t
df_ode[!, :N] = df_ode.S + df_ode.I + df_ode.R  # [illegible]
```

```
15-element Vector{Float64}:
 1000.0
 1000.0
  999.9999999999999
 1000.0
 1000.0
  999.9999999999999
 1000.0
  999.9999999999999
```

```
 999.9999999999999
1000.0
1000.0
1000.0
 999.9999999999999
1000.0
1000.0
```

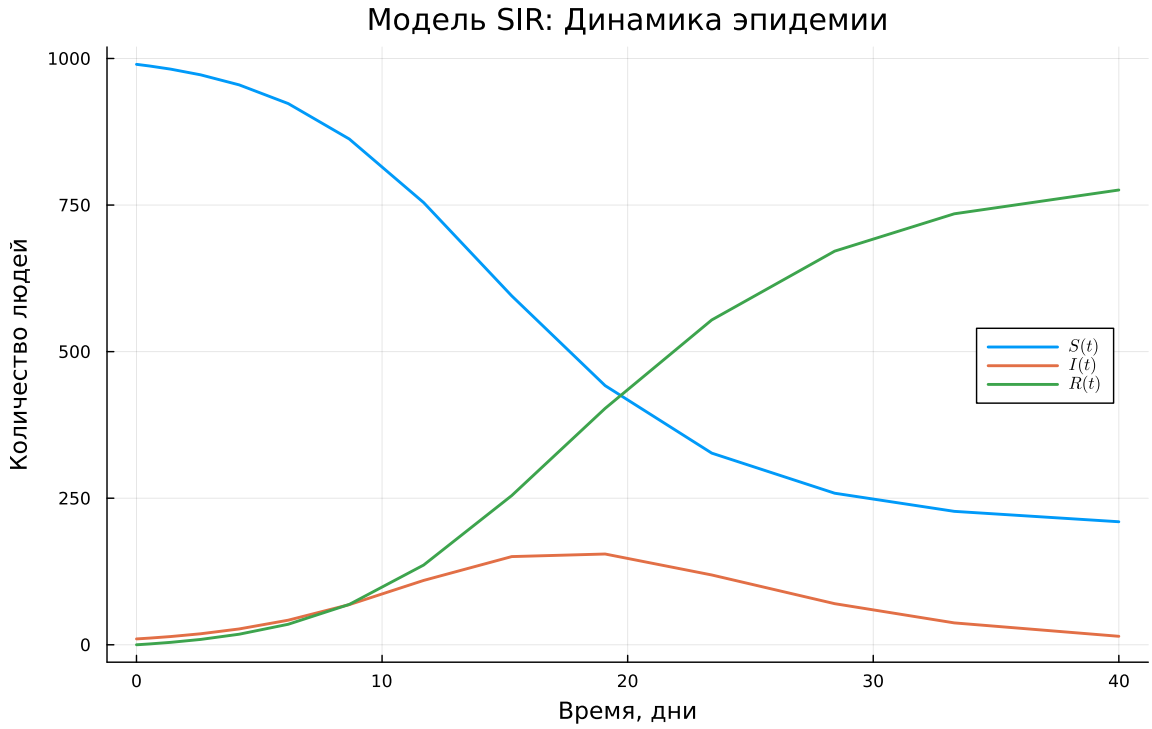
Вывод параметров модели

```
println("□□□□□□□□□ □□□□□□ SIR:")
println("□ (□□□□□□□□□□□ □□□□□□□□□) = ", p[1])
println("c (□□□□□□□ □□□□□ □□□□□□□□□) = ", p[2])
println("□ (□□□□□□□□ □□□□□□□□□□□□□) = ", p[3])
println("R₀ = c * □ / □ = ", round(R0, digits=3))
println("□□□□□□□ □□□□□□□□□□□□□□□□□ □□□□□□□ = ", round(1/p[3], digits=2), "
println("□□□□□□□□□ □□□□□□□□: S₀ = ", u0[1], ", I₀ = ", u0[2], ", R₀ = ",
```

```
□□□□□□□□□ □□□□□□ SIR:
□ (□□□□□□□□□□□ □□□□□□□□□) = 0.05
c (□□□□□□□ □□□□□ □□□□□□□□□) = 10.0
□ (□□□□□□□□ □□□□□□□□□□□□□) = 0.25
R₀ = c * □ / □ = 2.0
□□□□□□□ □□□□□□□□□□□□□□□□□ □□□□□□□ = 4.0 □□□□
□□□□□□□□□ □□□□□□□□: S₀ = 990.0, I₀ = 10.0, R₀ = 0.0
```

1. ОСНОВНОЙ ГРАФИК: динамика всех трех групп

```
plt1 = @df df_ode plot(:t,
    [:S :I :R],
    label=[L"S(t)" L"I(t)" L"R(t)"],
    xlabel="□□□□□, □□□",
    ylabel="□□□□□□□□□□ □□□□□",
    title="□□□□□□ SIR: □□□□□□□□ □□□□□□□□",
    linewidth=2,
    legend=:right,
    grid=true,
    size=(800, 500),
    fmt=:png)  # ← □□□□□□ □□□ □□□□□□
```



Добавление аннотаций с параметрами

```
annotate!(plt1, maximum(df_ode.t) * 0.7, maximum(df_ode.N) * 0.8,
    text("□□□□□□□□□:\n□ = $(p[1])\nc = $(p[2])\n□ = $(p[3])\nR₀ = $(ro
    8, :left))
```

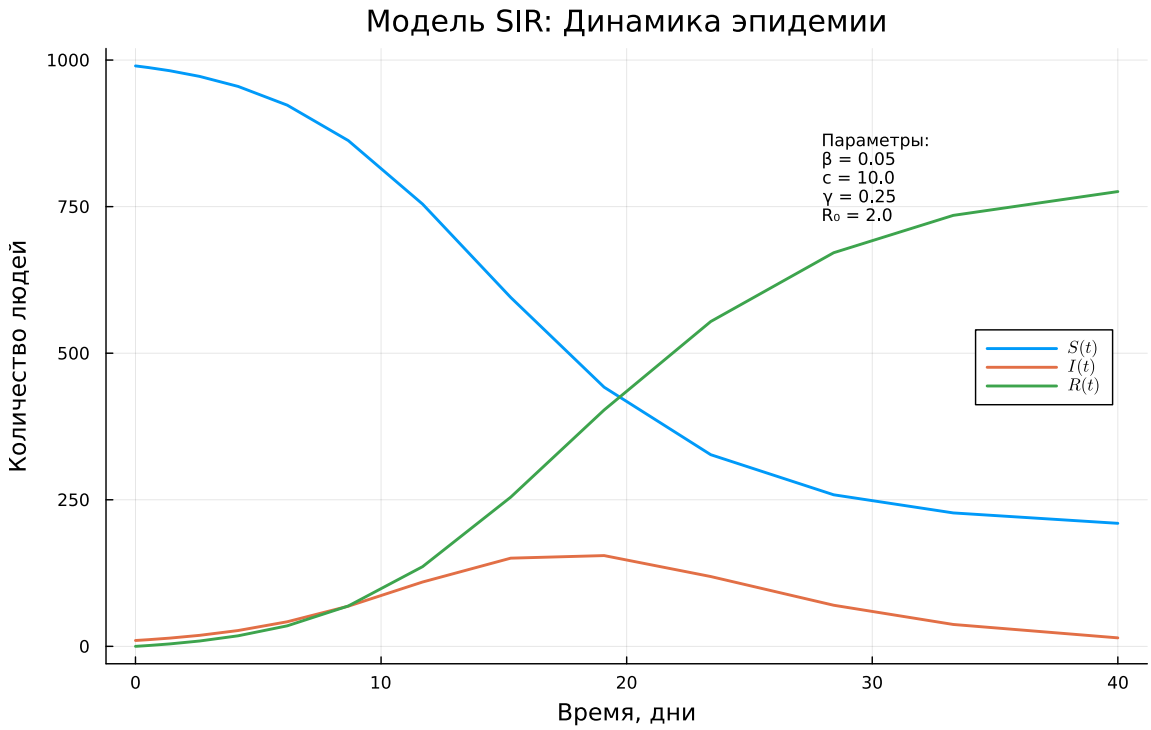
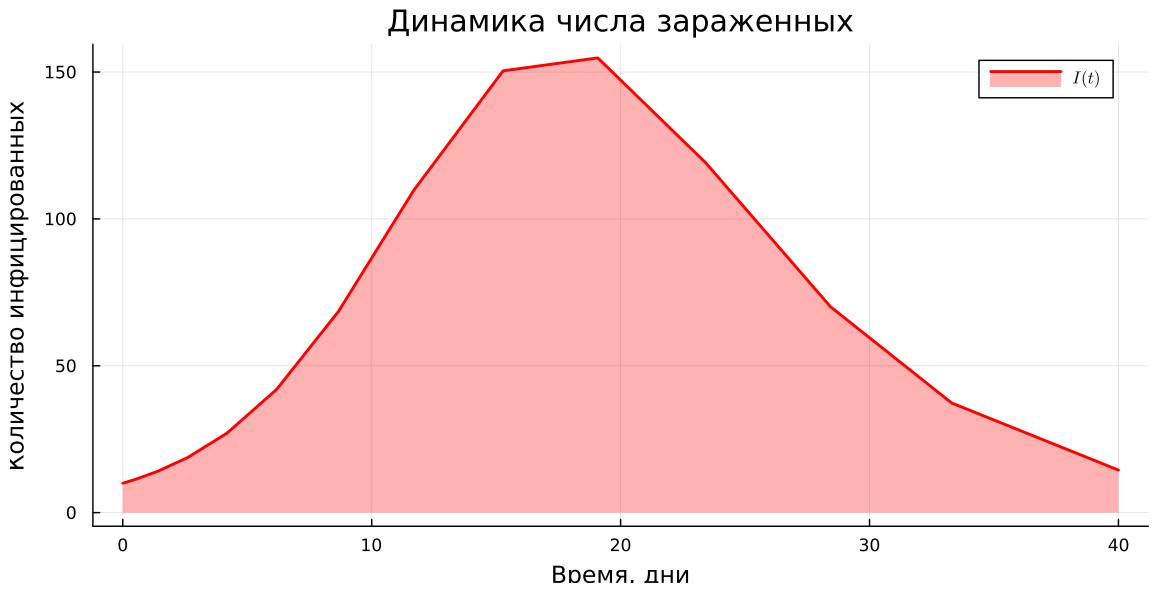


График только инфицированных (I)

```
plt2 = @df df_ode plot(:t, :I,
    label=L"I(t)",
    xlabel="□□□□□, □□□",
    ylabel="□□□□□□□□□□ □□□□□□□□□□□□□□",
    title="□□□□□□□□ □□□□□ □□□□□□□□□□□",
    color=:red,
    linewidth=2,
    fill=(0, 0.3, :red),
    grid=true,
    size=(800, 400),
    fmt=:png)
```



Отметка пика эпидемии

```
peak_idx = argmax(df_ode.I)
peak_time = df_ode.t[peak_idx]
peak_value = df_ode.I[peak_idx]
vline!(plt2, [peak_time], color=:black, linestyle=:dash, label=false
annotate!(plt2, peak_time, peak_value * 1.05,
    text("⌧⌧⌧: $(round(peak_value, digits=1)) ⌧⌧ $(round(peak_time, d
    8, :top))
```

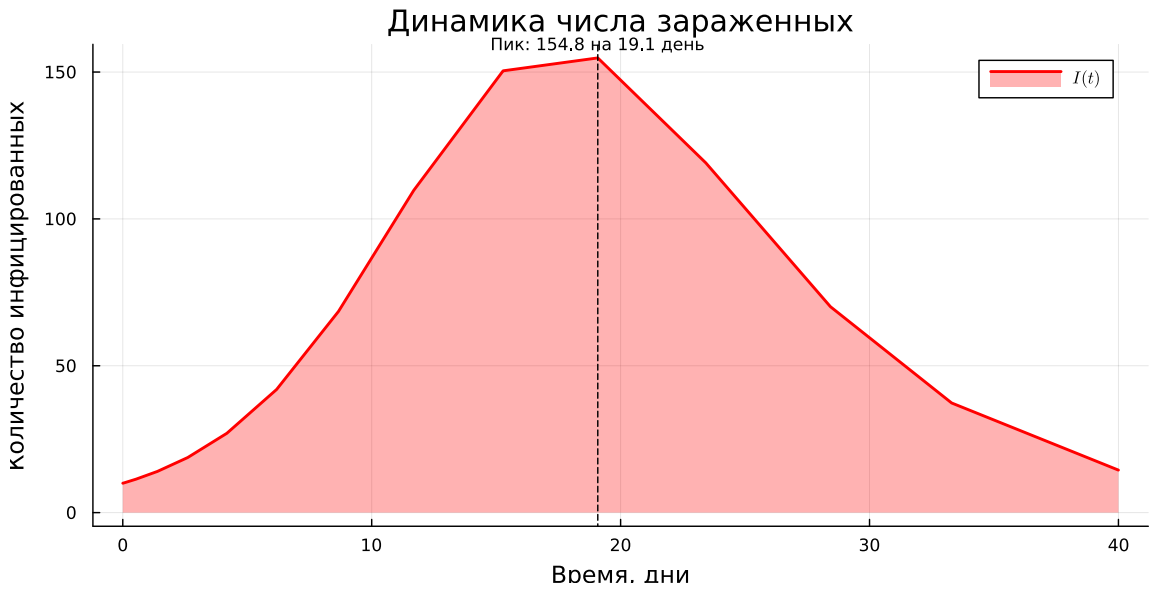


График в логарифмическом масштабе (для анализа экспоненциального роста)

```
plt3 = @df df_ode plot(:t, :I,
    label=L"I(t)",
    xlabel="□□□□□, □□□",
    ylabel="□□□□□□□□□□ □□□□□□□□□□□□□□ (□□□. □□□□□□□)",
    title="□□□□□□□□□□□□□□□□ □□□□ (□□□. □□□□□)",
    yscale=:log10,
    color=:red,
    linewidth=2,
    grid=true,
    size=(800, 400),
    fmt=:png)
```

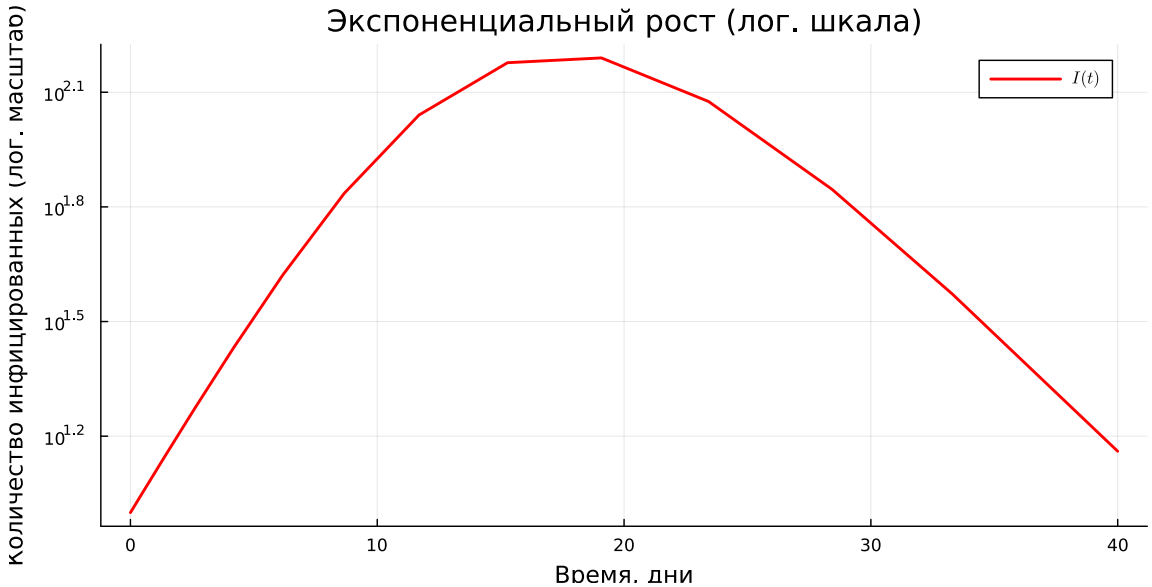
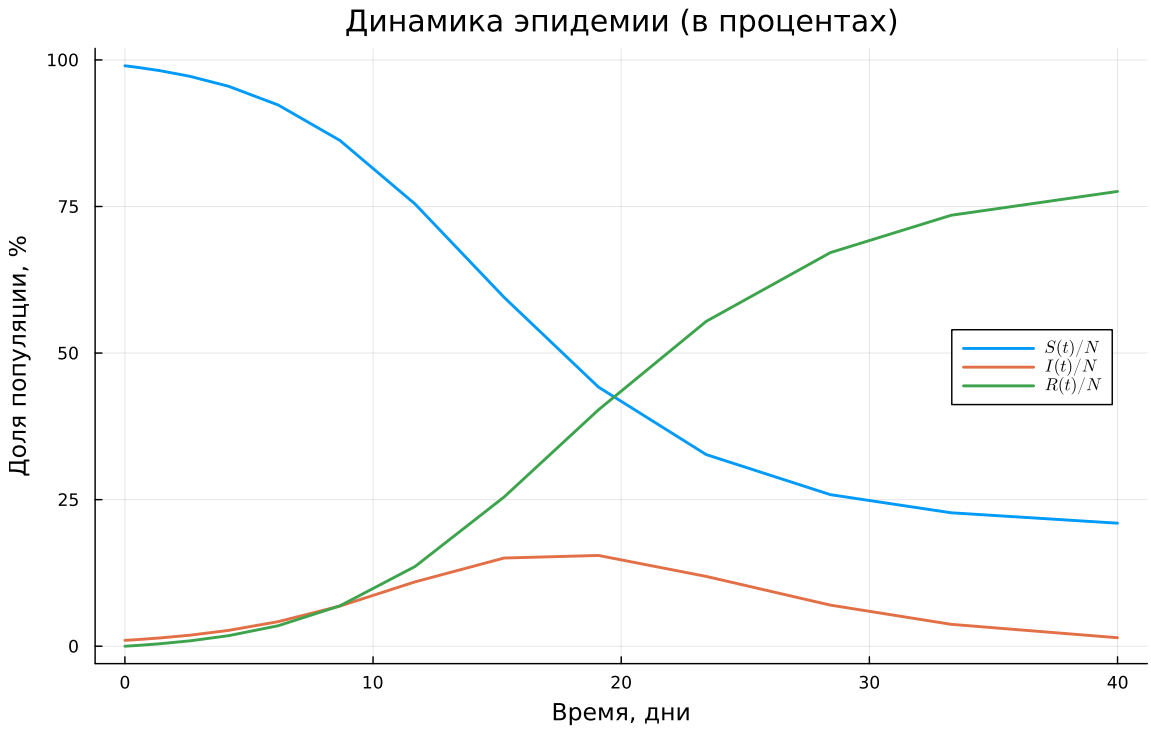


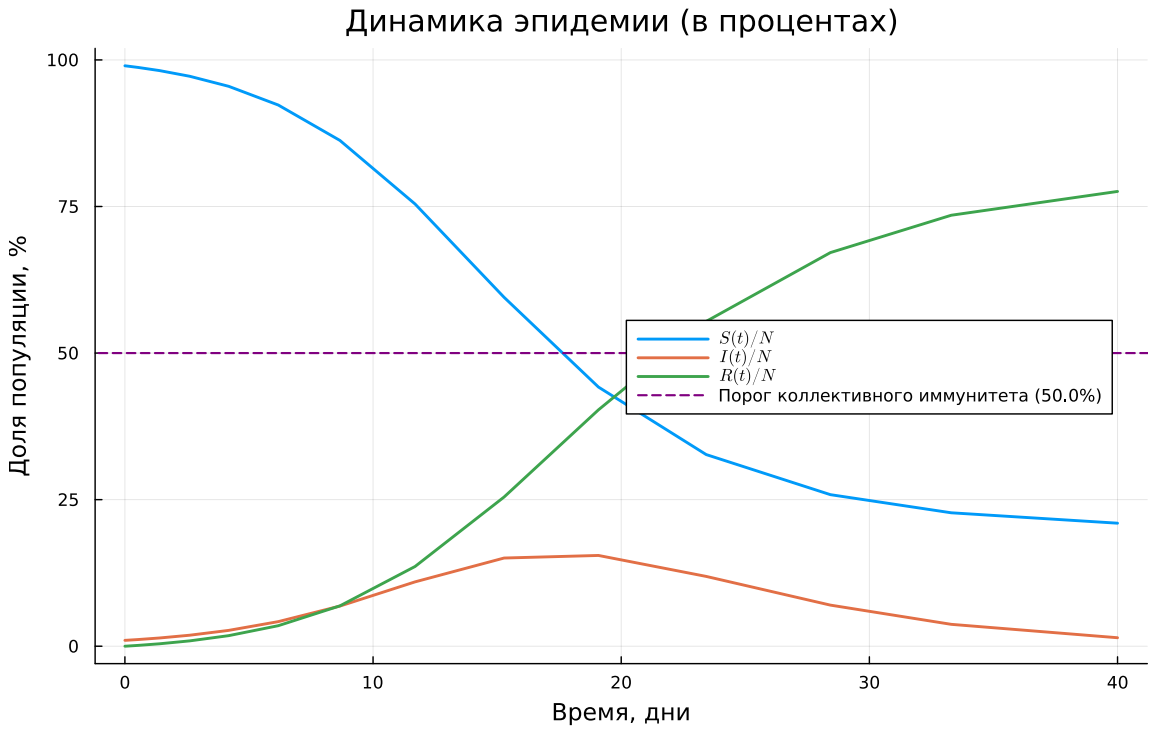
График долей населения (в процентах)

```
plt4 = @df df_ode plot(:t,
    [:S :I :R] ./ df_ode.N .* 100,
    label=[L"S(t)/N" L"I(t)/N" L"R(t)/N"],
    xlabel="□□□□□, □□□",
    ylabel="□□□□ □□□□□□□□□□, %",
    title="□□□□□□□□ □□□□□□□□□ (□ □□□□□□□□□□)",
    linewidth=2,
    legend=:right,
    grid=true,
    size=(800, 500),
    fmt=:png)
```



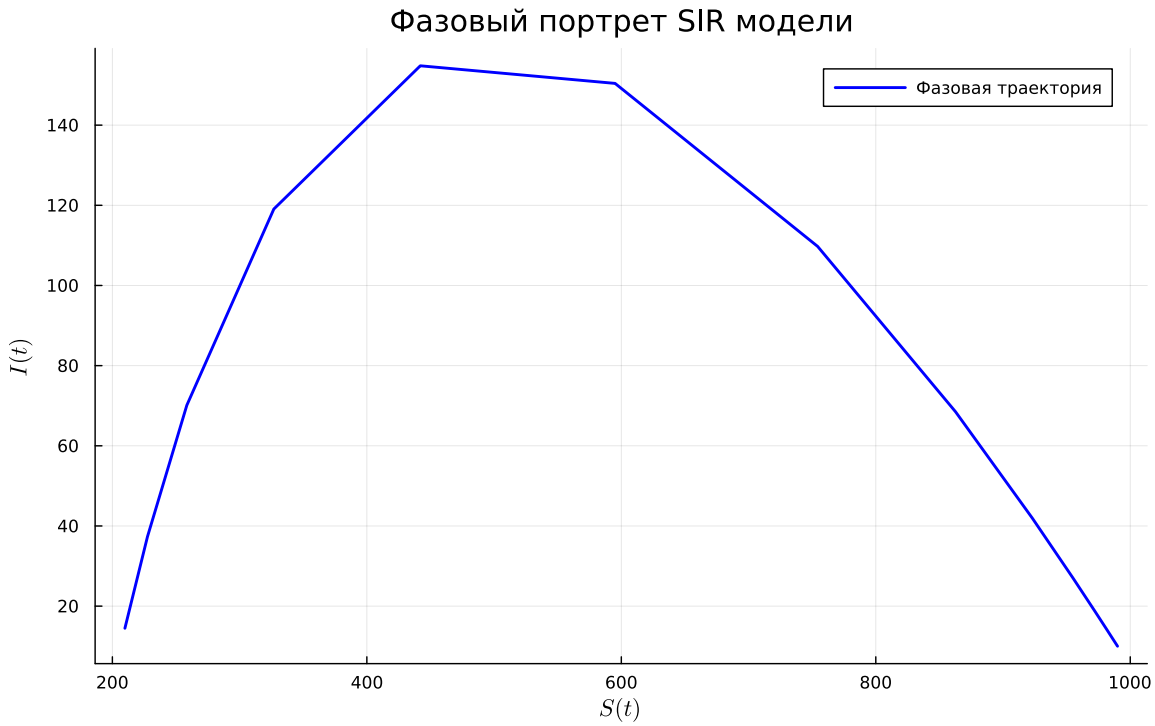
Горизонтальная линия для порога коллективного иммунитета

```
if R0 > 1
    herd_immunity_threshold = (1 - 1/R0) * 100
    hline!(plt4, [herd_immunity_threshold], color=:purple, linestyl
        label="□□□□□ □□□□□□□□□□□□ □□□□□□□□□□ ($(round(herd_immunity_thres
        linewidth=1.5)
end
```



Фазовый портрет (I vs S)

```
plt5 = plot(df_ode.S, df_ode.I,
    label="□□□□□□□ □□□□□□□□□□",
    xlabel=L"S(t)",
    ylabel=L"I(t)",
    title="□□□□□□□ □□□□□□□ SIR □□□□□□",
    color=:blue,
    linewidth=2,
    grid=true,
    size=(800, 500),
    fmt=:png,
    legend=:topright)
```



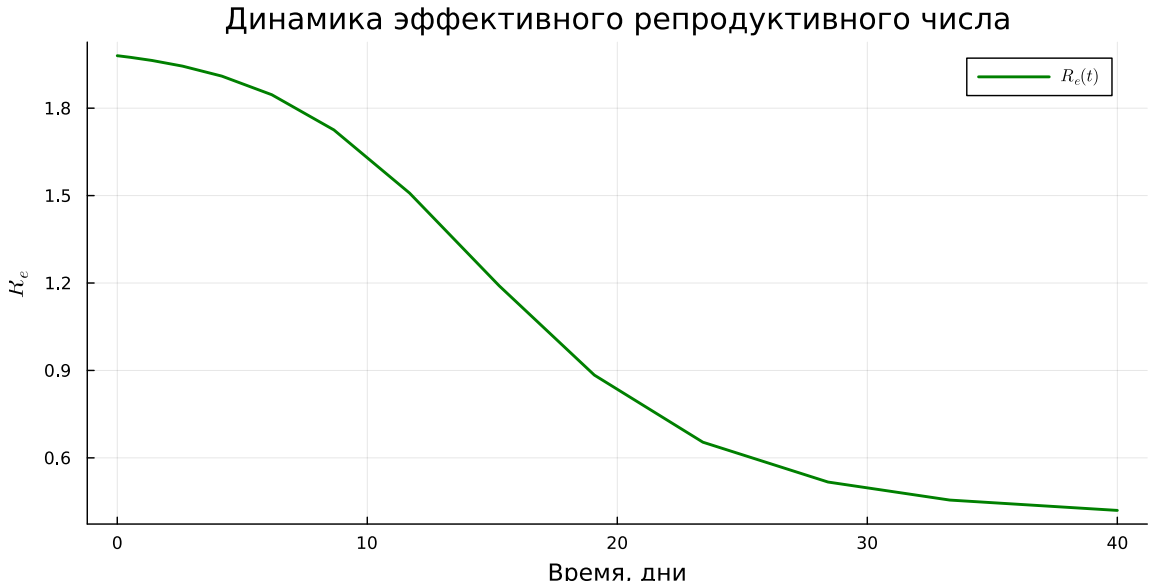
Добавление стрелок направления

```
for i in 1:50:length(df_ode.S)-1
    plot!(plt5, [df_ode.S[i], df_ode.S[i+1]], [df_ode.I[i], df_ode.
        arrow=:closed, color=:blue, alpha=0.5, label=false)
end
```

График R_e - эффективного репродуктивного числа

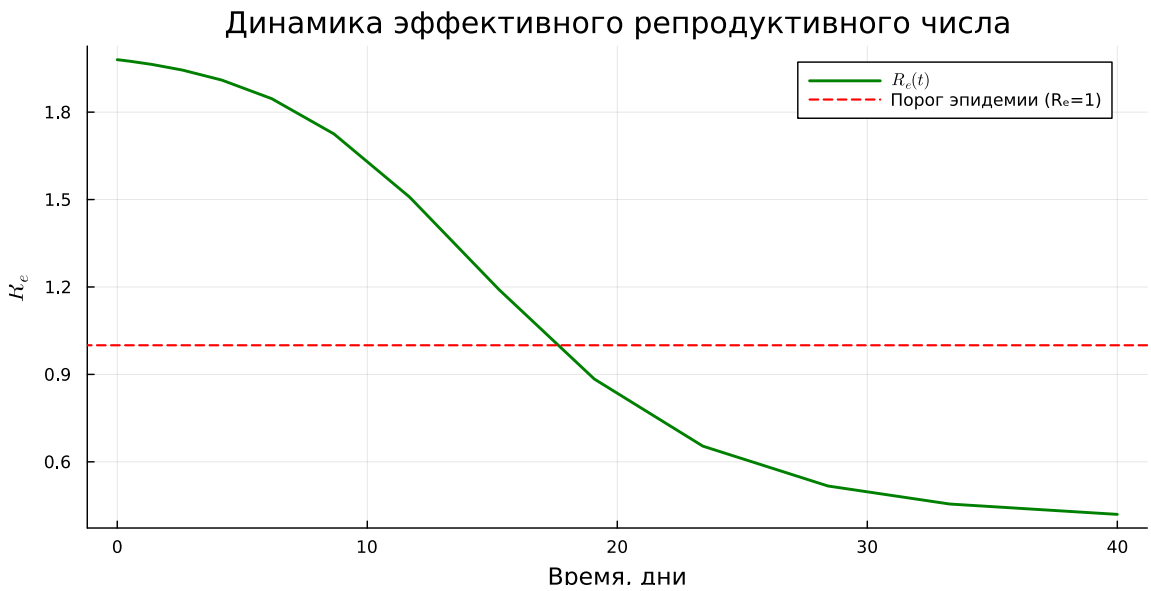
```
df_ode[!, :Re] = R0 .* df_ode.S ./ df_ode.N
plt6 = @df df_ode plot(:t, :Re,
    label=L"R_e(t)",
    xlabel="□□□□□, □□□",
    ylabel=L"R_e",
    title="□□□□□□□□ □□□□□□□□□□□□ □□□□□□□□□□□□□□□ □□□□□",
    color=:green,
    linewidth=2,
```

```
    grid=true,
    size=(800, 400),
    fmt=:png)
```



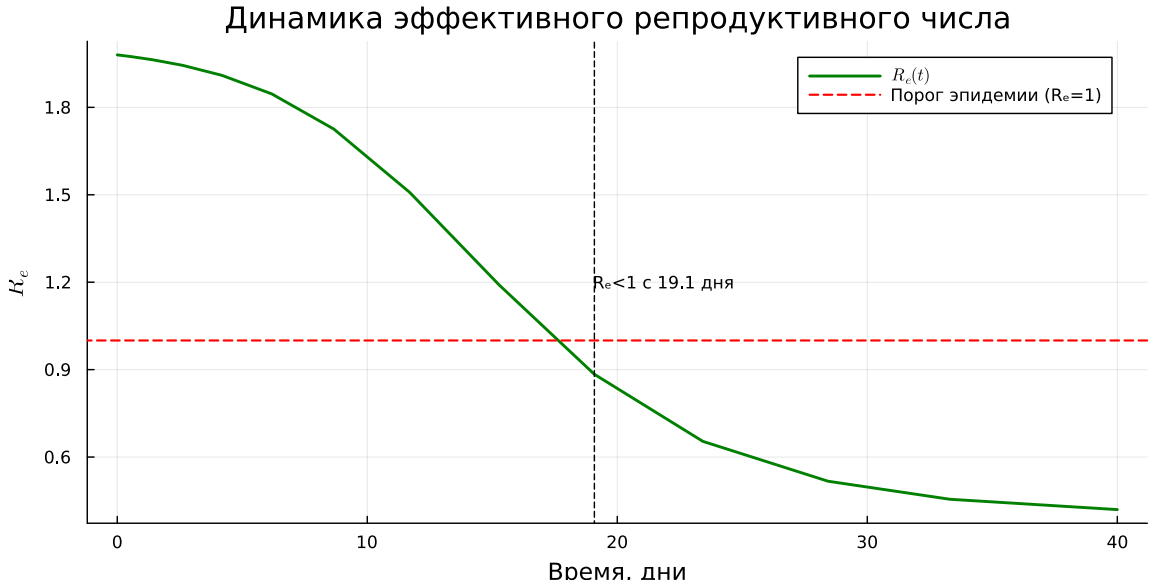
Горизонтальная линия на уровне 1

```
hline!(plt6, [1.0], color=:red, linestyle=:dash, label="
```



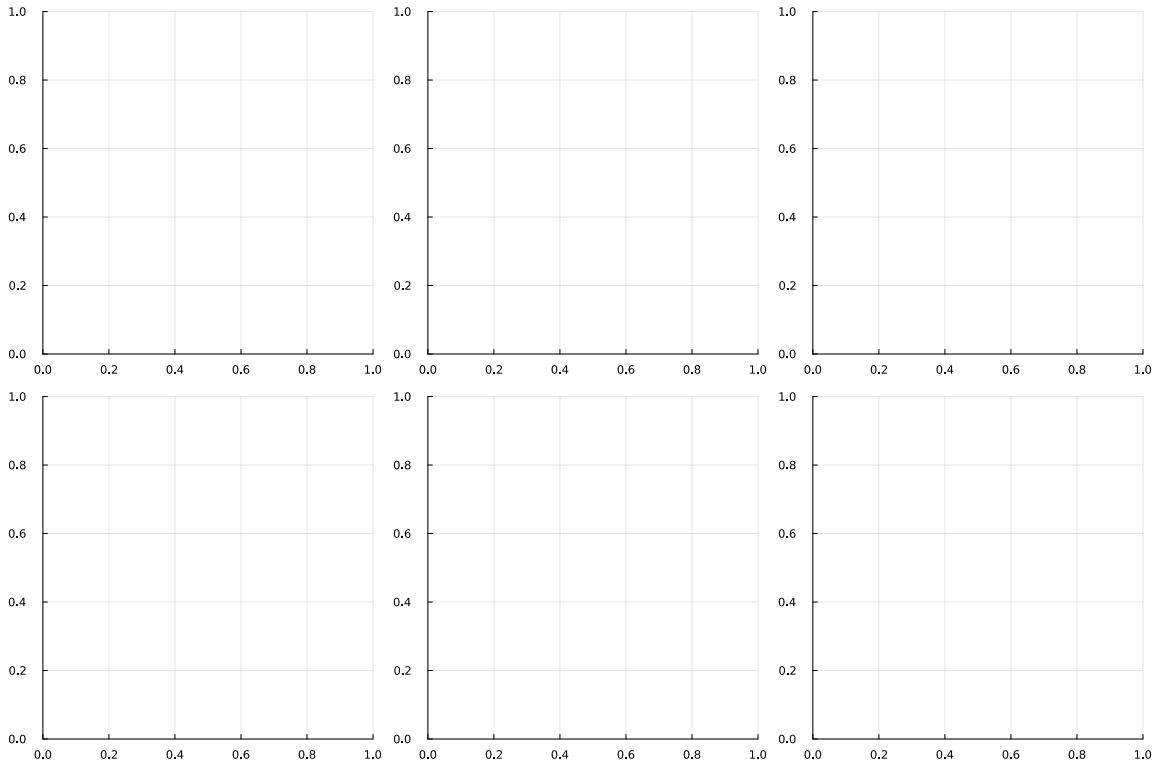
Отметка момента, когда R_e становится < 1

```
cross_idx = findfirst(x -> x < 1, df_ode.Re)
if !isnothing(cross_idx) && cross_idx > 1
    cross_time = df_ode.t[cross_idx]
    vline!(plt6, [cross_time], color=:black, linestyle=:dash, label
    annotate!(plt6, cross_time, 1.2,
        text("Rₑ<1 □ $(round(cross_time, digits=1)) □□□", 8, :left))
end
```



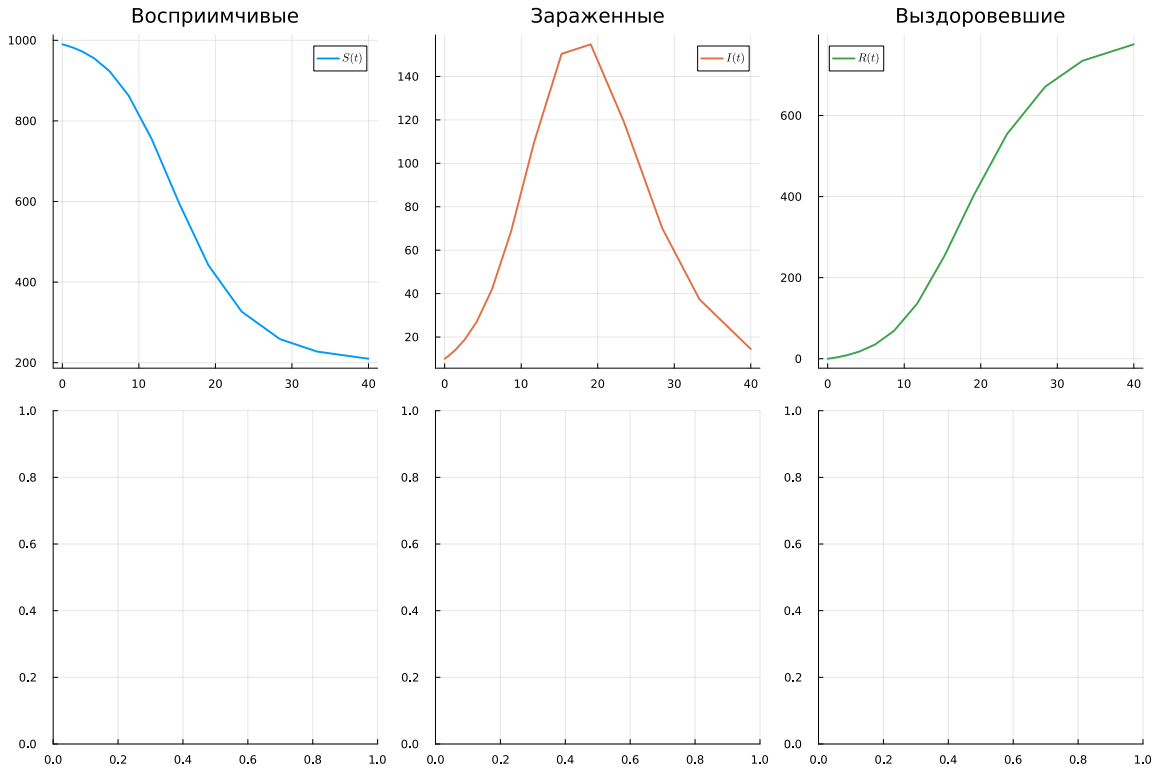
Компактный график всех кривых в одной панели

```
plt7 = plot(layout=(2, 3), size=(1200, 800),fmt=:png)
```



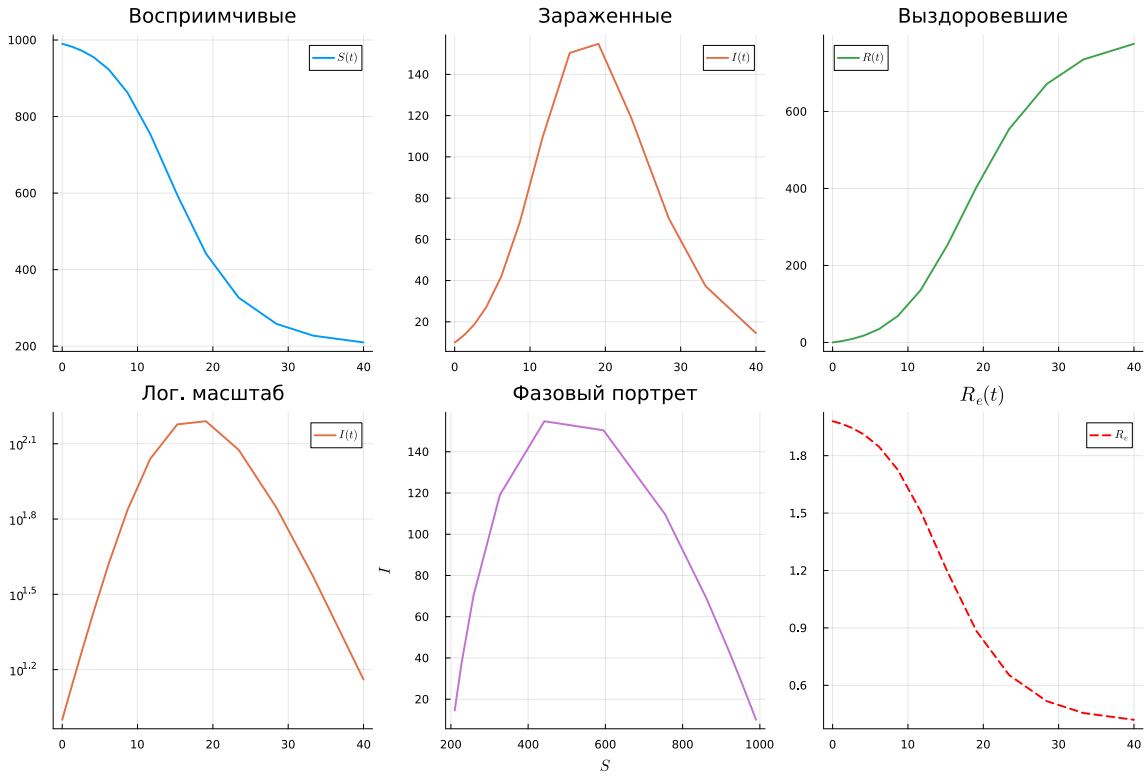
Верхний ряд

```
plot!(plt7[1], df_ode.t, df_ode.S, label=L"S(t)", color=1, linewidtl
plot!(plt7[2], df_ode.t, df_ode.I, label=L"I(t)", color=2, linewidtl
plot!(plt7[3], df_ode.t, df_ode.R, label=L"R(t)", color=3, linewidtl
```



Нижний ряд

```
plot!(plt7[4], df_ode.t, df_ode.I, label=L"I(t)", color=2, linewidt
    yscale=:log10, title="[illegible]. [illegible]")
plot!(plt7[5], df_ode.S, df_ode.I, label=false, color=4, linewidth=
    title="[illegible] [illegible]", xlabel=L"S", ylabel=L"I")
plot!(plt7[6], df_ode.t, df_ode.Re, label=L"R_e", color=:green, lin
    title=L"R_e(t)", hline=[1.0], linestyle=:dash, linecolor=:red)
```



Сохранение графиков

```
savefig(plt1, plotsdir(script_name, "sir_main.png"))
savefig(plt2, plotsdir(script_name, "sir_infected.png"))
savefig(plt3, plotsdir(script_name, "sir_log_scale.png"))
savefig(plt4, plotsdir(script_name, "sir_percentages.png"))
savefig(plt5, plotsdir(script_name, "sir_phase_portrait.png"))
savefig(plt6, plotsdir(script_name, "sir_effective_R.png"))
savefig(plt7, plotsdir(script_name, "sir_panel.png"))
```

```
"C:\\Users\\bogda\\work\\study\\2026-1\\2026-1--study--
simulationmodeling\\labs\\lab02\\project\\plots\\sir_panel.png"
```

Бенчмарк для оценки производительности

```
println("\n□□□□□□□□ □□□□□□□:")
@benchmark solve(prob_ode, dt = □t)
```

```
□□□□□□□□ □□□□□□□:

BenchmarkTools.Trial: 10000 samples with 1 evaluation per sample.
 Range (min … max):  72.800 □s …  8.264 ms  □ GC (min … max): 0.00% … 97.
 Time  (median):     100.900 □s               □ GC (median):    0.00%
 Time  (mean ± □):   109.729 □s ± 199.710 □s  □ GC (mean ± □):  5.26% ±  2.9

                        □□□▅□▂
  □□□□□□□□□□□□□□□▂□▅□□□□□□□□□□▂□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□ □
  72.8 □s         Histogram: frequency by time        151 □s <

 Memory estimate: 53.70 KiB, allocs estimate: 548.
```

Дополнительный анализ

```
println("\n=== □□□□□□ □□□□□□□□□□□ ===")
println("□□□□□ □□□□□□□□□□□ □□□□□□□□□ (□□□□□□□□): N = ", round(df_ode.N[1],
println("□□□□□□□ □□□□□ □□□□□□□□□□: I_max = ", round(peak_value, digits=1
println("□□□□□ □□□□□□□□□□ □□□□: t_peak = ", round(peak_time, digits=1),
println("□□□□□□□□ □□□□□ □□□□□□□□□□□□: R(∞) = ", round(df_ode.R[end], digi
println("□□□□ □□□□□□□□□□□□: ", round(df_ode.R[end]/df_ode.N[1]*100, dig

if R0 > 1
    println("\n□□□□□□□□□□□□□ □□□□□□:")
    println("  - □□□□□ □□□□□□□□□□□□□ □□□□□□□□□□: ", round((1-1/R0)*100, di
```

```
    println("  - □□□□□□□□□□□□□ □□□ □□□ S/N = 1/R₀ = ", round(1/R0, digit
end
```

```
=== □□□□□□ □□□□□□□□□□□ ===
□□□□□ □□□□□□□□□□□ □□□□□□□□□ (□□□□□□□□): N = 1000.0
□□□□□□□ □□□□□ □□□□□□□□□□: I_max = 154.8
□□□□□ □□□□□□□□□□ □□□□: t_peak = 19.1 □□□□
□□□□□□□□ □□□□□ □□□□□□□□□□□□: R(∞) = 775.7
□□□□ □□□□□□□□□□□□: 77.6%

□□□□□□□□□□□□□ □□□□□□:
  - □□□□□ □□□□□□□□□□□□□ □□□□□□□□□□: 50.0%
  - □□□□□□□□□□□□□ □□□ □□□ S/N = 1/R₀ = 0.5
```

5 Модель Лотки–Вольтерры

Все действия повторяю для нового скрипта lv_ode.jl.

Сам файл, описанный в литературном стиле:

```
using DrWatson
quickactivate(dirname(@__DIR__), "project")

using DifferentialEquations
using DataFrames
using StatsPlots
using LaTeXStrings
using Plots
using Statistics
using FFTW
using LinearAlgebra
using CSV

script_name = splitext(basename(PROGRAM_FILE))[1]
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))



# ## □□□□□□ □□□□□-□□□□□□□□□□ (□□□□□□-□□□□□□)
```

```
"""
□□□□□□ □□□□□-□□□□□□□□□ (□□□□□□-□□□□□□)

□□□□□□ □□□□□□ □□□□□□□□□ □□□□□□□□ □□□□□□□□□□□□□□ □□□□ □□□□□:
  - □□□□□□ (x) — □□□□□□□□□, □□□□□□□□ □□□□□ □□□ □□□□□□□□
  - □□□□□□□ (y) — □□□□□□□□□, □□□□□□□□□□ □□□□□□□□

□□□□□□□ □□□□□□□□□□□□□□□ □□□□□□□□□:
  dx/dt = □· x - □· x· y     (1)   — □□□□ □□□□□ □□□□□□□□□ □□□□□□□□□
  dy/dt = □· x· y - □· y     (2)   — □□□□ □□□□□□□□ □□□□□□□ □□ □□□□□□□ □□□□□

□□□□□□□□□□ □□□□□ □□□□□□□□□□:
  □ [□□□□□⁻¹] — □□□□□□□□□□□ □□□□□□□□□□□□□ □□□□□□□□ □□□□□
                  (□□□□□□□□□□□ □□□□□ □□□□□□□□□□□□ □□□□□□□□□□)
  □ [□□□□□⁻¹·□□□□□⁻¹] — □□□□□□□□□□□ □□□□□□□□□□□□□ □□□□□ □□□□□□□□
                  (□□□□□□□□□□□ □□□□□□□ □ □□□□□□□□)
  □ [□□□□□⁻¹·□□□□□⁻¹] — □□□□□□□□□□□ □□□□□□□□□ □□□□□□□□
                  (□□□□□□□ □□□□□□□□ □□□□□ □□□□□□□□□□ □□□□ □□□□□□)
  □ [□□□□□⁻¹] — □□□□□□□□□□□ □□□□□□□□□□□ □□□□□□□□□□ □□□□□□□□

□□□□□□□□□□□□ □□□□□□□:
  □□□□□□ □□□□ □□□□□□□□□□ □□□□□□□□□□ □□□□□□□□□ □□□□□□ (1925) □
  □□□□ □□□□□□□□□□ (1926). □□□□□□□□□ □□□□□□□□□□□ □□ □□□ □□□□□□□□□□
  □□□□□□□□□ □□□□□□ □□□ □ □□□□□□□□□□□□□□ □□□□□ □□□□□□ □□□□□□□ □□□□□.
"""
function lotka_volterra!(du, u, p, t)
    x, y = u                # x - □□□□□□□, y - □□□□□□□
    □, □, □, □ = p          # □□□□□□□□□ □□□□□□□
```

```
    @inbounds begin
        du[1] = □*x - □*x*y       # dx/dt: □□□□□□□□□ □□□□□□□□□ □□□□□
        du[2] = □*x*y - □*y       # dy/dt: □□□□□□□□□ □□□□□□□□□ □□□□□□□□
    end
    nothing
end


# ## □□□□□□□□□□□□□ □□□□□□□□ □□□□□□

"""
□□□□□□ □□□□□□□□□□□□□ □□□□□ □□□□□□ □□□□□-□□□□□□□□□□

□□□□□□□□ □□□□□ □□□ □□□□□□□□□□□□□ □□□□□:

1. □□□□□□□□□□□□ □□□□□: (x, y) = (0, 0)
   — □□□□□□□□□□□□□□ □□□□□□□□□ □□□□□ □□□□□.
   □□□□□□□□ □□□□□: J = [□  0; 0 -□]
   □□□□□□□□□□□□ □□□□□: □₁ = □ (>0), □₂ = -□ (<0)
    →□□□□□□□□□ □□□□□ (□□□□□□□□□□□□□)

2. □□□□□□□□□□□□□□ □□□□□: (x*, y*) = (□/□, □/□)
   — □□□□□ □□□□□□□□□□□□□□□ □□□□□.
   □□□□□□□□ □□□□□: J = [0  -□x*; □y*  0]
   □□□□□□□□□□□□ □□□□□: □□□□□ □□□□□□
    →□□□□□ (□□□□□□□□□□□ □□□□□□□□□□)

□ □□□□□□□□□□□□ □□□□□□□□□□□□□ □□□□□□□□□□□□□ □□□□□ □□□□□□□ □□□□□ □□□:
```

```
  x(t) ≈ x* + A·cos(ωt + φ)
  y(t) ≈ y* + B·sin(ωt + φ)

ωωω ωωωωωωω ωωωωωωωωω: ω = √(αβ)
ωωωωωω ωωωωωωωωω: T = 2π/√(αβ)
"""
function analyze_fixed_points(p)
    α, β, γ, δ = p

    # ωωωωωωωωωωωω ωωωωω ωωωωωωωωωωωωωω
    x_star = γ / δ
    y_star = α / β

    # ωωωωωωω ω ωωωωωω ωωωωωωωω
    ω = sqrt(α * β)
    T = 2π / ω

    # ωωωωωωωω ωωωωωωωωωωω (ωωωωωωωωωωω)
    J = [0  -β*x_star; α*y_star  0]
    eigenvalues = eigvals(J)

    return (;
        x_star = x_star,
        y_star = y_star,
        frequency = ω,
        period = T,
        eigenvalues = eigenvalues,
        is_stable = all(real(eigenvalues) .<= 0)
```

```
    )
end


# ## □□□□□□ □□□□□□□□□□□ □□□ □□□□□□□□□□□

"""
□□□□□□ □□□□□□□□□□□ □□□□□□ □□□□□-□□□□□□□□□□

□□□□□□ □□□□□ □□□□□□□□□□□□□ □□□□□□□□□ □□□□□□□□□□□□□ □□□□□□□□□:
  1. □□□□□□□□□□□□□ □□□□□□□□ — □□□□□□□ □□□□□□□□□ □□ □□□□□□□□□
  2. □□□□□□□ □□□□□□□□ — □□□□□□□ □□□□□□□□ □□□□□□□□□□□ □□□□□
  3. □□□□□□□□□ □□□□□□□□ — □□□□□□ □□□□□□□□ □□□□□□□□□□□ □□□□□
"""
parameter_sets = [
    # [□, □, □, □]
    [0.1, 0.02, 0.01, 0.3],     # □□□□□ 1: □□□□□□□□□□□□□ (□□□□□□□□□ □□□□□□□□□
    [0.2, 0.02, 0.01, 0.3],     # □□□□□ 2: □□□□□□□ □□□□ □□□□□ (□□□□□□□ □□□□□
    [0.05, 0.02, 0.01, 0.3],    # □□□□□ 3: □□□□□□□□□ □□□□ □□□□□ (□□□□□□ □□□□
    [0.1, 0.04, 0.02, 0.3],     # □□□□□ 4: □□□□□□□ □□□□□□□□□□□□□ □□□□□□□□
    [0.1, 0.01, 0.005, 0.15]    # □□□□□ 5: □□□□□□□□□□□□□□□□□ (□□□□□□□□□□ □□□□
]

parameter_names = [
    "□□□□□□□□□□□□□ (□□□□□□□□□ □□□□□□□□□)",
    "□□□□□□□ □□□□ □□□□□ (□□□□□□□ □□□□□□□)",
    "□□□□□□□□□ □□□□ □□□□□ (□□□□□□ □□□□□□□)",
    "□□□□□□□ □□□□□□□□□□□□□ □□□□□□□□",
```

```
    "□□□□□□□□□□□□□□□□"
]

# □□□□□□□□□ □□□□□□□ (□□□□□□□□□□ □□□ □□□□ □□□□□□□□□□□□□)
u0_lv = [40.0, 9.0]  # [□□□□□□, □□□□□□□]

# □□□□□□□□□ □□□□□□□□□
tspan_lv = (0.0, 200.0)    # □□□□□□□□□□□□ □□□□□□□□□
dt_lv = 0.01               # □□□ □□□□□□□□□□□□□□□


# ## □□□□□□□ □□□ □□□□□□□ □□□□□□ □ □□□□□□□□□ □□□□□□□□□□□

"""
    solve_lotka_volterra(p, u0, tspan; kwargs...)

□□□□□□ □□□□□□□ □□□□□-□□□□□□□□□ □ □□□□□□□□□ □□□□□□□□□□□.

□□□□□□□□□:
  p — □□□□□□ □□□□□□□□□□ [□, □, □, □]
  u0 — □□□□□□□□□ □□□□□□□ [x0, y0]
  tspan — □□□□□□□□ □□□□□□□ (t_start, t_end)

□□□□□□□□□□ □□□□□□□□□ □ □□□□□□□□ □ □□□□□□□□□□□□□□□□□□□□ □□□□□□□.
"""
function solve_lotka_volterra(p, u0, tspan; saveat=0.1, reltol=1e-8
    # □□□□□□□□ □ □□□□□□□ □□□□□□
    prob = ODEProblem(lotka_volterra!, u0, tspan, p)
```

```
    sol = solve(prob,
                Tsit5(),
                dt = dt_lv,
                reltol = reltol,
                abstol = abstol,
                saveat = saveat,
                dense = true)

    # □□□□□□□□□□ □□□□□□
    df = DataFrame()
    df[!, :t] = sol.t
    df[!, :prey] = [u[1] for u in sol.u]
    df[!, :predator] = [u[2] for u in sol.u]

    # □□□□□□ □□□□□□□□□□□
    □, □, □, □ = p
    df[!, :dprey_dt] = □ .* df.prey .- □ .* df.prey .* df.predator
    df[!, :dpredator_dt] = □ .* df.prey .* df.predator .- □ .* df.pr

    # □□□□□□□□□□□□□ □□□□□□□□□
    df[!, :prey_rel_change] = df.dprey_dt ./ df.prey .* 100
    df[!, :predator_rel_change] = df.dpredator_dt ./ df.predator .*

    return (sol = sol, df = df)
end


# ## □□□□□□ □□□□□□□□□□□ □□□ □□□□ □□□□□□□ □□□□□□□□□□
```

```
println("\n" * "="^80)
println(" " * "□□□□□□ □□□□□-□□□□□□□□□: □□□□□□ □□□□□□□ □□□□□□□□□□")
println("="^80)

# □□□□□□□□ □□□□□□□□□□□ □□□□ □□□□□□□□□□□□□
results = []

for (idx, p) in enumerate(parameter_sets)
    □, □, □, □ = p

    println("\n" * "-"^60)
    println("□□□□□□□□□□□ #$idx: $(parameter_names[idx])")
    println("-"^60)

    # □□□□□□□□□□□□□ □□□□□□ □□□□□□□□□□□□ □□□□□
    fp_analysis = analyze_fixed_points(p)
    println("\n□ □□□□□□□□□□□□□ □□□□□□:")
    println("  □□□□□□□□□□□□ □□□□□: (x*, y*) = (",
            round(fp_analysis.x_star, digits=2), ", ",
            round(fp_analysis.y_star, digits=2), ")")
    println("  □□□□□□□□□□□□□ □□□□□□□ □□□□□□□□□: □ = ", round(fp_analysis.f
    println("  □□□□□□□□□□□□□ □□□□□□: T = ", round(fp_analysis.period, di

    # □□□□□□□□□ □□□□□□□
    println("\n□ □□□□□□□□□ □□□□□□□□□□□□□□:")

    @time begin
        result = solve_lotka_volterra(p, u0_lv, tspan_lv)
```

```
    sol, df = result.sol, result.df
end

# □□□□□□□□□□□□□□ □□□□□□
prey_min = minimum(df.prey)
prey_max = maximum(df.prey)
prey_mean = mean(df.prey)
predator_min = minimum(df.predator)
predator_max = maximum(df.predator)
predator_mean = mean(df.predator)

println("\n□ □□□□□□□□□□ □□□□□□□□□:")
println("  □□□□□□: min = ", round(prey_min, digits=2),
        ", max = ", round(prey_max, digits=2),
        ", □□□□□□□ = ", round(prey_mean, digits=2))
println("  □□□□□□□: min = ", round(predator_min, digits=2),
        ", max = ", round(predator_max, digits=2),
        ", □□□□□□□ = ", round(predator_mean, digits=2))

# □□□□□□□□□□□□ □□□□□□ (□□□□□□□□□□ □□□□□□□□□□□□ □□□□□□□)
function find_dominant_frequency(signal, dt)
    n = length(signal)
    spectrum = abs.(rfft(signal .- mean(signal)))
    freq = rfftfreq(n, 1/dt)

    # □□□□ □□□□□□□□, □□□□□□□□ □□□□□□□ □□□□□□□
    idx_max = argmax(spectrum[2:end]) + 1
    return freq[idx_max], spectrum[idx_max]
```

```
    end

    dom_freq_prey, _ = find_dominant_frequency(df.prey, dt_lv)
    dom_freq_pred, _ = find_dominant_frequency(df.predator, dt_lv)

    println("\n□ □□□□□□□□□□□□ □□□□□□:")
    println("  □□□□□□□□□□□□ □□□□□□□ (□□□□□□): ", round(dom_freq_prey, dig
    println("  □□□□□□□□□□□□□□□□□ □□□□□□ (□□□□□□): ", round(1/dom_freq_prey
    println("  □□□□□□□□□□□□ □□□□□□□ (□□□□□□□): ", round(dom_freq_pred, di
    println("  □□□□□□□□□□□□□□□□□ □□□□□□ (□□□□□□□): ", round(1/dom_freq_pre

    # □□□□□□□□ □□□□□□□□□□□□ □□□□□□
    period_error = abs(1/dom_freq_prey - fp_analysis.period) / fp_a
    println("\n□ □□□□□□□□□□□□ □□□□□□:")
    println("  □□□□□□□□□□ □□□□□□□ □□ □□□□□□□□□□□□□: ", round(period_error,

    # □□□□□□□□□□ □□□□□□□□□□□
    push!(results, (;
        params = p,
        name = parameter_names[idx],
        df = df,
        sol = sol,
        stats = (min_prey=prey_min, max_prey=prey_max, mean_prey=pr
                 min_pred=predator_min, max_pred=predator_max, mean
        fp = fp_analysis,
        dom_freq = (prey=dom_freq_prey, predator=dom_freq_pred)
    ))
```

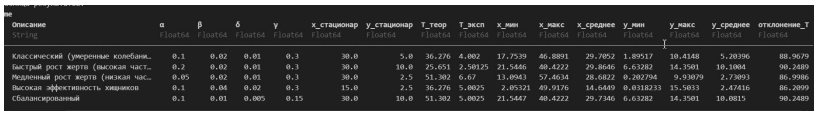
```
    # □□□□□□□□□□ □□□□□□□ □□□ □□□□□□□□ □□□□□□ □□□□□□□□□□
    plt = plot(df.t, [df.prey df.predator],
               label=[L"□□□□□□ (x)" L"□□□□□□□ (y)"],
               xlabel="□□□□□", ylabel="□□□□□□□□□",
               title="□□□□□-□□□□□□□□□: $(parameter_names[idx])",
               linewidth=2,
               legend=:topright,
               size=(900, 400))

    hline!(plt, [fp_analysis.x_star], color=:green, linestyle=:dash
    hline!(plt, [fp_analysis.y_star], color=:red, linestyle=:dash, 

    savefig(plt, plotsdir(script_name, "lv_set$(idx)_dynamics.png")
end

# ## □□□□□□□□□□□□□ □□□□□□ □□□□ □□□□□□□ □□□□□□□□□□

println("\n" * "="^80)
println(" " * "□□□□□□□□□□□□□ □□□□□□ □□□□□□□ □□□□□□□□□□")
println("="^80)

# □□□□□□□ □□□□□□□ □□□□□□□
comparison_df = DataFrame(
    □□□□□ = Int[],
    □□□□□□□□ = String[],
    □ = Float64[],
    □ = Float64[],
    □ = Float64[],
```

```
    □ = Float64[],
    x_□□□□□□□□□ = Float64[],
    y_□□□□□□□□□ = Float64[],
    T_□□□□ = Float64[],
    T_□□□□ = Float64[],
    x_□□□ = Float64[],
    x_□□□□ = Float64[],
    x_□□□□□□□ = Float64[],
    y_□□□ = Float64[],
    y_□□□□ = Float64[],
    y_□□□□□□□ = Float64[],
    □□□□□□□□□□_T = Float64[]
)

for (idx, r) in enumerate(results)
    push!(comparison_df, [
        idx,
        r.name,
        r.params[1],
        r.params[2],
        r.params[3],
        r.params[4],
        r.fp.x_star,
        r.fp.y_star,
        r.fp.period,
        1/r.dom_freq.prey,
        r.stats.min_prey,
        r.stats.max_prey,
```

```
        r.stats.mean_prey,
        r.stats.min_pred,
        r.stats.max_pred,
        r.stats.mean_pred,
        abs(1/r.dom_freq.prey - r.fp.period) / r.fp.period * 100
    ])
end

println("\n□ □□□□□□□ □□□□□□□ □□□□□□□□□□□:")
println(comparison_df)

# □□□□□□□□□□ □□□□□□□ □□□□□□□
CSV.write(datadir(script_name, "comparison_results.csv"), compariso


# ## □□□□□□□□□□□□ □□□□□□□□□ □□□□ □□□□□□□

# □□□□□□ 1: □□□□□□□□□ □□□□□□□□ □□□□□□□□□
plt_periods = bar(1:length(results),
                  [r.fp.period for r in results],
                  label="□□□□□□□□□□□□□ □□□□□□",
                  fillalpha=0.5,
                  xlabel="□□□□□ □□□□□□ □□□□□□□□□□",
                  ylabel="□□□□□□ □□□□□□□□□",
                  title="□□□□□□□□□ □□□□□□□□ □□□□□□□□□")

bar!(plt_periods, 1:length(results),
     [1/r.dom_freq.prey for r in results],
```

```
    label="□□□□□□□□□□□□□□□□□ □□□□□□ (□□□□□□)",
    fillalpha=0.5)

savefig(plt_periods, plotsdir(script_name, "comparison_periods.png"

# □□□□□□ 2: □□□□□□□□□□ □□□□□□□□□□□□ □□□□□
x_stars = [r.fp.x_star for r in results]
y_stars = [r.fp.y_star for r in results]

plt_fixed = scatter(x_stars, y_stars,
                    label="□□□□□□□□□□□□ □□□□□",
                    xlabel=L"x* (□□□□□□□□□ □□□□□)",
                    ylabel=L"y* (□□□□□□□□□ □□□□□□□□)",
                    title="□□□□□□□□□□□□□ □□□□□□ □□□ □□□□□□ □□□□□□□□ □□□□□□□□□□",
                    markersize=8,
                    markercolor=:blue,
                    grid=true)

for (idx, (x, y)) in enumerate(zip(x_stars, y_stars))
    annotate!(plt_fixed, x, y, text("  $idx", :left, 10))
end

savefig(plt_fixed, plotsdir(script_name, "comparison_fixed_points.p
```

результаты(рис. 5.1)



Описание	α	β	δ	γ	x_стационар	y_стационар	T_теор	T_эксп	x_мин	x_макс	x_среднее	y_мин	y_макс	y_среднее	отклонение_T
String	Float64	Float64	Float64	Float64	Float64	Float64	Float64	Float64	Float64	Float64	Float64	Float64	Float64	Float64	Float64
Классический (умеренные колебани…	0.1	0.02	0.01	0.3	30.0	5.0	36.276	4.002	17.7539	46.8891	29.7052	1.89517	10.4148	5.20396	88.9079
Быстрый рост жертв (высокая част…	0.2	0.02	0.01	0.3	30.0	10.0	25.651	2.50125	21.5446	40.4222	29.8646	6.63282	14.3501	10.1004	90.2489
Медленный рост жертв (низкая час…	0.05	0.02	0.01	0.3	30.0	2.5	51.302	6.67	13.0943	57.4634	28.6822	0.202794	9.93079	2.73093	86.9986
Высокая эффективность хищников	0.1	0.04	0.02	0.3	15.0	2.5	36.276	5.0025	2.05321	49.9176	14.6849	0.0318233	15.5033	2.47416	86.2099
сбалансированный	0.1	0.01	0.005	0.15	30.0	10.0	51.302	5.0025	21.5447	40.4222	29.7346	6.63282	14.3501	10.0815	90.2489

Рисунок 5.1: notebook работает

5.1 МОДЕЛЬ ЛОТКИ-ВОЛЬТЕРРЫ (ХИЩНИК-ЖЕРТВА)

using DrWatson quickactivate(dirname(__**DIR**__), «project»)

using DifferentialEquations using DataFrames using StatsPlots using LaTeXStrings using Plots using Statistics using FFTW using LinearAlgebra using CSV

script_name = «lv_ode» mkpath(plotsdir(script_name)) mkpath(datadir(script_name))

“ “ ” Модель Лотки-Вольтерры (хищник-жертва)

Данная модель описывает динамику взаимодействия двух видов: - Жертвы (x) — популяция, служащая пищей для хищников - Хищники (y) — популяция, питающаяся жертвами

Система дифференциальных уравнений: dx/dt = α·x - β·x·y (1) — рост жертв ограничен хищниками dy/dt = δ·x·y - γ·y (2) — рост хищников зависит от наличия жертв “ “ ” function lotka_volterra!(du, u, p, t) x, y = u α, β, δ, γ = p **inbounds** begin du[1] = *αx - βxy du[2] = δxy - γy* end nothing end

“ “ ” Анализ стационарных точек модели Лотки-Вольтерры “ “ ” function analyze_fixed_points(p) α, β, δ, γ = p x_star = γ / δ y_star = α / β ω = sqrt(α * γ) T = 2π / ω J = [0 -β*x_star; δ*y_star 0] eigenvalues = eigvals(J) return (; x_star = x_star, y_star = y_star, frequency = ω, period = T, eigenvalues = eigenvalues, is_stable = all(real(eigenvalues) .<= 0)) end

“ “ ” Наборы параметров модели Лотки-Вольтерры “ “ ” parameter_sets = [[0.1, 0.02, 0.01, 0.3], # Набор 1: Классический [0.2, 0.02, 0.01, 0.3], # Набор 2: Быстрый рост жертв [0.05, 0.02, 0.01, 0.3], # Набор 3: Медленный рост жертв [0.1, 0.04, 0.02, 0.3], # Набор 4: Высокая эффективность хищников [0.1, 0.01, 0.005, 0.15] # Набор 5: Сбалансированный]

parameter_names = [«Классический», «Быстрый рост жертв», «Медленный рост жертв», «Высокая эффективность хищников», «Сбалансированный»]

u0_lv = [40.0, 9.0] tspan_lv = (0.0, 200.0) dt_lv = 0.01

“ “ ” solve_lotka_volterra(p, u0, tspan; kwargs...) “ “ ” function solve_lotka_volterra(p, u0, tspan; saveat=0.1, reltol=1e-8, abstol=1e-10) prob = ODEProblem(lotka_volterra!, u0, tspan, p) sol = solve(prob, Tsit5(), dt=dt_lv, reltol=reltol, abstol=abstol, saveat=saveat, dense=true)

```
df = DataFrame()
df.t = sol.t
df.prey = [u[1] for u in sol.u]
df.predator = [u[2] for u in sol.u]

□, □, □, □ = p
df.dprey_dt = □ .* df.prey .- □ .* df.prey .* df.predator
df.dpredator_dt = □ .* df.prey .* df.predator .- □ .* df.predator
df.prey_rel_change = df.dprey_dt ./ df.prey .* 100
df.predator_rel_change = df.dpredator_dt ./ df.predator .* 100

return (sol = sol, df = df)
```

end

6 Основной цикл вычислений (весь вывод закомментирован)

results = []

for (idx, p) in enumerate(parameter_sets) α, β, δ, γ = p

```
# [illegible] [illegible]
fp_analysis = analyze_fixed_points(p)

# [illegible] [illegible]
result = solve_lotka_volterra(p, u0_lv, tspan_lv)
sol, df = result.sol, result.df

# [illegible]
prey_stats = (min=minimum(df.prey), max=maximum(df.prey), mean=mean(d
pred_stats = (min=minimum(df.predator), max=maximum(df.predator), mea

# [illegible] [illegible]
function find_dominant_frequency(signal, dt)
    n = length(signal)
    spectrum = abs.(rfft(signal .- mean(signal)))
    freq = rfftfreq(n, 1/dt)
    idx_max = argmax(spectrum[2:end]) + 1
```

```
    return freq[idx_max], spectrum[idx_max]
end

dom_freq_prey, _ = find_dominant_frequency(df.prey, dt_lv)
dom_freq_pred, _ = find_dominant_frequency(df.predator, dt_lv)

# ⊠⊠⊠⊠⊠⊠⊠⊠⊠⊠ ⊠⊠⊠⊠⊠⊠⊠⊠⊠⊠
push!(results, (;
    params = p,
    name = parameter_names[idx],
    fp = fp_analysis,
   dom_freq = (prey=dom_freq_prey, predator=dom_freq_pred),
    stats = (prey=prey_stats, pred=pred_stats)
))

# ⊠⊠⊠⊠⊠⊠
plt = plot(df.t, [df.prey df.predator],
           label=[L"⊠⊠⊠⊠⊠⊠ (x)" L"⊠⊠⊠⊠⊠⊠⊠ (y)"],
           xlabel="⊠⊠⊠⊠⊠", ylabel="⊠⊠⊠⊠⊠⊠⊠⊠⊠",
           title="⊠⊠⊠⊠⊠-⊠⊠⊠⊠⊠⊠⊠⊠⊠: $(parameter_names[idx])",
        linewidth=2, legend=:topright, size=(900, 400), fmt=:png)
hline!(plt, [fp_analysis.x_star], color=:green, linestyle=:dash, labe
hline!(plt, [fp_analysis.y_star], color=:red, linestyle=:dash, label=
savefig(plt, plotsdir(script_name, "lv_set$(idx)_dynamics.png"))

  end
```

7 Сравнительный анализ

comparison_df = DataFrame(Набор = 1:length(results), Описание = [r.name for r in results], α = [r.params[1] for r in results], β = [r.params[2] for r in results], δ = [r.params[3] for r in results], γ = [r.params[4] for r in results], x_стационар = [r.fp.x_star for r in results], y_стационар = [r.fp.y_star for r in results], T_теор = [r.fp.period for r in results], T_эксп = [1/r.dom_freq.prey for r in results], x_мин = [r.stats.prey.min for r in results], x_макс = [r.stats.prey.max for r in results], x_среднее = [r.stats.prey.mean for r in results], y_мин = [r.stats.pred.min for r in results], y_макс = [r.stats.pred.max for r in results], y_среднее = [r.stats.pred.mean for r in results], отклонение_T = [abs(1/r.dom_freq.prey - r.fp.period) / r.fp.period * 100 for r in results])

CSV.write(datadir(script_name, «comparison_results.csv»), comparison_df)

8 Графики сравнения

plt_periods = bar(1:length(results), [r.fp.period for r in results], label=«Теоретический период», fillalpha=0.5, fmt=:png) bar!(plt_periods, 1:length(results), [1/r.dom_freq.prey for r in results], label=«Экспериментальный период», fillalpha=0.5) savefig(plt_periods, plotsdir(script_name, «comparison_periods.png»))

plt_fixed = scatter([r.fp.x_star for r in results], [r.fp.y_star for r in results], label=«Стационарные точки», markersize=8, fmt=:png) savefig(plt_fixed, plotsdir(script_name, «comparison_fixed_points.png»))

9 Выводы

Я познакомился с мождельюSIR, описывающей распространение болезни, и с моделью Лотки–Вольтерры, описывающей популяцию хищников и жертв. Перевёл код в литературный стиль, а также написал программу подбора значений.

Список литературы